

Homework 2

For this assignment, you will implement and submit code following these specifications. Most are the same as Homework 1, but there are some updates. I tried to highlight them all as changes.

You will also submit a report. See below for more details, but the core of the report will discuss the process of moving from HW1 to HW2, so keep notes on how your final code differs from your plans and drafts from HW1, if any, and why you made those changes.

Please hand your code and report in in a zip file.

The project: an Ngram language model

Your code should include a corpus reader for reading in text files, a Python class for building an n-gram model from the data, code for generating text from the model, and a comparative analysis of different models you build.

Throughout, you may add any of your own modules, methods, attributes, classes, variables, and functions that you like. Do not, however, change the names or signatures of given functions etc., except, if you like, to add optional parameters. That is, we your graders should be able to run our tests.

Include comments for any code that is at all complicated or hard to understand. When in doubt, comment.

Give your variables, functions, etc. interpretable names. You may use single letters (traditionally e.g. `i`, `j`) for indexing, and things like `n`, `m` for the “n” of “n-gram”, but otherwise, avoid variable names like `x`. If you define new classes, use `CamelCase` for their names; otherwise use `lower_case_with_underscores`.

Every module (yes, the file itself), class, function, and method requires a docstring. Some are started for you, but require updating.

Your final project should have all `TODO` comments implemented, and the `TODO` markers removed.

The `corpus_reader` module

In Practicum 4, you build a class called `CorpusReader`. The file called `corpus_reader.py` has the beginnings of a similar class. You can use your code from class to get started.

Here we define ~~two~~ **three** classes. HW1 mentioned a `Tokenizer` that turns text into sentences and words, and a `CorpusReader` that reads in a corpus and uses an instance of a `Tokenizer` to tokenize it.

For HW2, you will also implement a *subclass* of `Tokenizer` called `TokenizerNLTK` that uses NLTK’s `word_tokenize` and `sent_tokenize`.

Tokenizer

Update the given `Tokenizer` class by completing the given methods.

Private Method `_clean_word` By convention, methods that are not meant to be used outside of the class are called private, and are named with an underscore at the beginning. Unlike some languages, Python doesn't treat them any differently, but the notation is useful to keep track of what you intend to do with the methods, and some IDEs will give warnings if you use a private method outside the class.

This method should remove punctuation at the start and end of a string, but leave internal punctuation in place.

e.g. `\ "Wow!!!\ "` -> `Wow`

e.g. `can't` -> `can't`

Method `sent_tokenize` Takes as argument a string and returns a list of strings. The method should split up the input text on periods (`"."`), question marks, and exclamation marks, followed by any number of whitespace characters.

Here you may choose whether to keep the punctuation for now, or discard it.

For example, `"Here is my sentence. 'What a nice sentence it is!!!' \tWow!\n"` could return:

```
['Here is my sentence', 'What a nice sentence it is', 'Wow']
```

or

```
"Here is my sentence.", "'What a nice sentence it is!!!'", "Wow!"]
```

Method `word_tokenize` Takes as argument a string and returns a list of strings. The method should split up the input text on any number of whitespace characters, and call `_clean_word` on each word.

For example, `"Here is my sentence. 'What a nice sentence it is!!!' \tWow!\n"` should return:

```
['Here', 'is', 'my', 'sentence', 'What', 'a', 'nice', 'sentence',  
'it', 'is', 'Wow']
```

Method `string2sentences` Takes as argument a string and returns a list of lists of strings. Uses `sent_tokenize` and `word_tokenize` to get tokenized sentences.

For example, `"Here is my sentence. 'What a nice sentence it is!!!' \tWow!\n"` should return:

```
[['Here', 'is', 'my', 'sentence'], ['What', 'a', 'nice', 'sentence',  
'it', 'is'], ['Wow']]
```

Class `TokenizerNLTK`

Inherits from your `Tokenizer` class, meaning it will act just like the `Tokenizer` class except in ways that you tell it to behave differently.

Your subclass should have its declaration as follows:

```
class TokenizerNLTK(Tokenizer):
```

This subclass only needs to implement the `sent_tokenize` and `word_tokenize` methods. Everything else will be automatically inherited from `Tokenizer`, and whenever it needs either of these two methods, it will use the one from `TokenizerNLTK`. That is, your new methods “override” the ones in `Tokenizer`.

NLTK’s tokenizers do not require the `_clean_word` method. Your new methods just need to call NLTK’s `word_tokenize` and `sent_tokenize` as appropriate, and throw away any empty words or words that consist entirely of punctuation that result.

Note that the `CorpusReader` class takes as argument a `Tokenizer` (which can include a `TokenizerNLTK` object), so when you implement a corpus reader, you can choose both corpus and its tokenizer.

Class `CorpusReader`

Initialiser Should store the directory path in the attribute `path` and the tokenizer in the attribute `tokenizer`, but should not read in any files.

New HW2 Method `raw` Reads in the corpus and returns a single string of everything concatenated together.

Method `sents` Reads in the corpus and returns a list of list of strings. Each inner list is a sentence of the reader’s corpus, split into its words. Use the tokenizer’s `string2sentences` method to split up the corpus.

Method `words` Reads in the corpus and returns all of its words as a list of strings. Use the tokenizer’s `word_tokenize` methods to split up the corpus.

HW2 addition: `tests.py` script

This file will hold definitions and calls of test functions for your code.

Here you will see just the requirements for the `Tokenizer` class tests.

When the script is run as a script, it should run the tests.

NB for advanced students: These can just be functions, not proper unit tests, but it’s up to you.

Tokenizer test functions

Write two functions to test your two `Tokenizer` classes.

Names: `test_tokenizer_plain` and `test_tokenize_nltk`.

Hand-write a tiny corpus of two or three sentences. Include punctuation that you hope to be removed, and punctuation you hope to keep. Include an instance of “n’t” as in “didn’t”.

Hand-write what the output of `Tokenizer.string2sentences` should be on that tiny corpus. Store that in a variable as well. The function should return `True` if applying the function has the same output, and `False` if not. If `False`, it should also print out a helpful message.

Once you have this working, and your code is passing the test, use the same tiny corpus to test your `TokenizerNLTK` class. You should find that it treats “n’t” differently, but otherwise it will likely be identical, and should at least have the same format.

Use these tests to help you ensure your classes meet the specifications in the assignment regarding the output of `string2sentences`.

Add function calls to these tests in your script. Include some printing so users can see what is being tested and whether it passed the test.

The model module

The provided file `model.py` is the model module started for you, with provided methods and partially-complete methods. Your job will be to complete it, including the documentation.

This module defines a class `NGramModel` that builds an ngram table from the sentences or words of a corpus. It can also calculate the perplexity of a given string, and generate sample strings from the model.

The initialiser takes the optional parameter `mode`. If it is set to `"sentence"`, the model’s atoms should be words, and it should model sentences. If it is set to `"word"`, the model’s atoms should be characters, and it should model words. The class is set up so that you have to change very little to move between these two modes: setting the mode and providing the right structure of data should be enough.

When used for sentences, you will provide to the model the data you get from the `CorpusReader`’s `sents` method. When you use it to model words, you will give it the output of the `words` method.

The description of the model class uses the terms “prefix” and “successor” a lot. A prefix is all but the last element of an ngram, and the successor of the prefix is the last element of that ngram.

The `self.successors` attribute of the model is all elements that are ever successors to any prefix. This should be all the words/characters, plus the end marker, but not the start marker.

Instructions

Complete the `NGramModel` class by implementing/completing all missing/incomplete methods.

Module includes global variables `START = "<s>"` and `END = "</s>"`

Initialiser Signature: `__init__(self, data, n, tokenizer, mode="sentence")`

The `data` is as read in by a `CorpusReader` `sents` or `words` method.

When initialised, the class should:

- Define attributes `n` (int), `tokenizer` (a `Tokenizer`), and `mode` ("sentence" or "word").
- Check that `mode` is either `sentence` or `word`, and if not, throw a `ValueError` with a helpful message.
- Find all ngrams and their frequencies. Store them as attribute `ngram_table`, a dict from prefixes to dicts from successors to counts.
- **Not** store the raw data! Just the ngram table.

Important note: Dictionaries can't have lists as their keys, because they're not "hashable". To get around this, you are provided with a method `hashable_prefix` that gives you a hashable key for any prefix.

Method preprocess The data read in by the corpus reader may need further processing.

Signature `preprocess(self, item)`

An `item` is a sentence (sentence mode) or word (word mode). Preprocesses a single data point:

- make all characters lowercase
- if in word mode, make item into a list of its characters
- Add the appropriate number of `START` and `END` tokens to the item.
- returns a list of strings.

Method probability Use the ngrams stored in `self.ngram_table` to compute the probability of an ngram. Apply LaPlace smoothing if called for.

Signature `probability(self, prefix, w_n, smoothing_constant=0)`

Computes the probability of element `w_n` following the `prefix`. If `smoothing_constant > 0`, use it for LaPlace smoothing.

Method `perplexity` Process the datapoint the same way you process each item when you built the model. Calculate its perplexity. Apply LaPlace smoothing if called for.

Note: it's recommended you do this in log space to avoid floating point errors, but this is not required.

Signature `perplexity(self, item, smoothing_constant=0)`

Computes the perplexity of `item` (tokenized sentence or word), passing the smoothing constant on to the `probability` function.

New HW2 Method `corpus_perplexity` Use the model's `Tokenizer` to calculate the total perplexity of a corpus.

Signature `corpus_perplexity(self, corpus: str, smoothing_constant=0)`

The argument `corpus` is a string, so apply the appropriate tokenizer method (`word_tokenize` or `strings2sentences`) and then call the model's `perplexity` method on each item, passing on the smoothing constant.

Method `get_successors` To use the language model generatively, we need to find which elements can come next, and what their frequencies are.

Signature `get_successors(self, prefix)`

Gets all possible successors to a prefix, paired with their counts.

Tests

Add to `tests.py` the following tests:

`test_ngram_table` For a very tiny corpus of words, and a very tiny corpus of sentences, write a function that initialises a trigram model ($n=3$). Compare the stored `ngram_table` with a hand-written correct dictionary. Return `True` if it's the same and `False` if not, together with a helpful message.

`test_probability_zero` Using your tiny model from `test_ngram_table`, test that you get the following:

1. 4 parts: Raw AND smoothed probability 0 for a prefix OR successor not in the model at all
2. For a prefix in the model and a successor that is in the model but not in that prefix's successors:
 1. 0 raw prob
 2. non-zero smoothed prob

Module **generate**

Defines functions to use a trained model generatively.

Demonstrates several models with a script.

Required: set random seed to 42

```
import random
random.seed(42)
```

This will make the outputs the same each time, making it easier to trouble-shoot and evaluate. (You can temporarily comment it out or change it while you're working if you like.)

(if you use a different randomiser, please set its seed to 42 as well.)

HW2 update: Required function: `generate_one`:

Generates one sentence **or word, depending on the mode**, using the probabilities defined in the model. Starts with start token(s), and uses `random.choices` or a similar function to choose probabilistically from the possible successors to the words generated so far.

**** New HW2** Required function: `demonstrate`:**

Signature: `demonstrate(model)`

Demonstrates one model as follows:

1. Print the models' string representation using e.g. `print(my_model)` (not `my_model.print_model()`, as this is way too much printing!)
2. Generate 5 words/sentences using your function `generate_one` and print them out nicely.
 - Note: these should be the same each time if you set the random seed.
3. Calculate and print the raw perplexity of the 6 test words/sentences below using the model's `corpus_perplexity` method on EACH word/sentence. That is, pretend each word/sentence is its own corpus. (This is because of words like "Don't" – see if you can understand why!)
4. Similarly, Calculate and print the LaPlace-smoothed perplexity with `smoothing_constant=1` for the 6 words/sentences

Make sure your printing is very well structured.

Test words for word-mode:

1. "swicky"
2. "these"
3. "devils"
4. "foonch"

5. “FAQ”
6. “isn’t”

For your looping convenience, here they are as a list: ["swicky", "these", "devils", "foonch", "FAQ", "isn't"]

Test sentences for sentence-mode:

1. “Don’t make too much noise,” said Father Brown.
2. Don’t go there!
3. Father Brown laughed.
4. make noise too much don’t Father said Brown.
5. The men left.
6. The swicky men left.

For your looping convenience, here they are as a list: ["\nDon't make too much noise,\n" said Father Brown.", "Don't go there!", "Father Brown laughed.", "make noise too much don't Father said Brown.", "The men left.", "The swicky men left."]

As an example of pretty-printing, my solution prints something like this:

```
-----
2-gram model for sentences
Tokenizer class: TokenizerNLTK
V = 13964
Number of prefixes: 13964
-----
Generated items:
blah blah
blah blah blah blah blah
...
-----
Perplexities:
"Don't make too much noise," said Father Brown.
    Raw:          xx.yy
    Smoothed:     xx.yy
...
-----
```

HW2 NEW: Script

When imported as a module, `generate.py` should not run any code: it should just provide definitions. When run as a script, it should do the following things.

Create a script section, protected by `if __name__ == "__main__":`.

Give your script a command-line interface. Users should be able to control (at least) the training data, the size of the ngrams, and the mode (sentence or word).

It's okay if you, for instance, can only control the training data if you also control the mode.

If the script is run with the argument “-h”, print instructions for how to use it.

That is, `python generate.py -h` should result in printed instructions for using the script.

Build and demonstrate the requested model using the `demonstrate` function.

HW2 New: `demo.py` script

Demonstrate your code by training and demonstrating 24 different models as follows.

Everything here should be as nicely-formatted as possible. Add spaces, labels, tabs, etc to help your reader navigate the output.

You will probably find that doing everything here will take a little while, depending on the speed of your computer. Mine is quite fast and altogether takes around 1.5 minutes. Test your code with smaller corpora and smaller n-grams if things are slow. Just make sure everything is in the code in the end when you hand it in.

Corpus Readers Initialise a `Tokenizer` and a `TokenizerNLTK`.

For each tokenizer, initialise a `CorpusReader` with the data provided in `train.zip`.

Train 12 models Create 12 ngram models with the following parameters:

1. `n` = 2, 3, and 4
2. `tokenizer` set to the plain one and the NLTK one
3. `mode` set to "sentence" and "word"

The `data` for each should then be from the `words` or `sents` methods of the appropriate corpus reader.

For example, one of the 12 should be a bigram model over letters in words, using `TokenizerNLTK`. It will use the `words` method from the corpus reader with `TokenizerNLTK` to get `data`, and also have its own tokenizer parameter set to that tokenizer. Its `n` parameter will be 2 and `mode="word"`.

Hint: you may find it convenient to embed for-loops and make the models programmatically, e.g.

```
for n in [2, 3, 4]:
    for mode in ["sentence", "word"]:
        ...
```

Use the models For each of the 12 models:

1. Run the `demonstrate` function.
2. Copy the 5 words/sentences into a table in your report (see report details below).
3. Copy the perplexity results into a table in your report.

Gutenberg Corpus: 12 more models Finally, train six word and six sentence models using NLTK's Gutenberg corpus as follows:

1. `n = 2, 3, 4`
2. tokenizer: always your `TokenizeNLTK` tokenizer
3. data:
 1. the output of the `sents()` or `words()` method of the NLTK Gutenberg Corpus as appropriate
 2. take the output of the `raw()` method of the NLTK Gutenberg Corpus and apply your Tokenizer's `string2sentences` or `word_tokenize` method, as appropriate.

Use the `demonstrate` function. Add the results to your report tables.

Report

Write up your report and submit it as either a PDF, a text file, or a Markdown file. Format it nicely to make it easy to read.

Give your report a name and add your names and IDs and the date.

Your report should accomplish three main things:

1. Describe the process of your work, including how you worked together.
2. Comment on the relationship between your HW1 and HW2 submissions.
3. Present and discuss the results from the `generate.py` script.

You can organise the report as you like, but here I will explain the requirements in three parts.

Process report

Describe your group's methods for doing this project, including HW1. Include in your answer:

1. To what extent did you split up the work and to what extent did you work together?
2. Generally, who did what?
3. Were there any group-work difficulties (a sick member, difficulty dividing the work fairly, etc.)?
4. Did you use Git? If so, how did it go?

HW1 to HW2

In HW1 you made a plan and in HW2 you implemented it. In theory, anyway! But this is probably not really what you did. Maybe you already started writing code in HW1. Maybe you realised between HW1 and HW2 that you had something important completely wrong. Maybe there were some things you didn't know how to do yet, and now you do.

For each module, discuss the relationship between your HW1 discussion and what you are handing in for HW2.

Also include in your answer comments on the following: in HW1 you were asked to answer some questions. Did answering or trying to answer them in advance, before HW2, help you with HW1 and/or HW2?

Script outputs

Your **demo** script trains 24 models and generates example words/sentences and calculates perplexities for each.

You are asked to put the data into tables in your report. Organise your perplexity table(s) so that you can discuss the following questions with reference to your results.

(Note that there are questions asking you whether there is an effect of A on B. You don't need to run statistical tests or anything, just comment on your impression of the data.)

1. Are perplexities generally higher or lower with LaPlace smoothing? Can you make a generalisation here, or is it too varied?
2. Under what circumstances is one of the smoothed and unsmoothed perplexities infinite, while the other is not? (Which is infinite in these cases? is it always the same one?)
 1. Why?
3. Under what circumstances are both smoothed and unsmoothed perplexities infinite?
4. Do perplexities get (monotone) higher or lower as n increases?
 1. Why?
5. Is there an effect of the tokeniser on perplexity? Does it depend on the corpus?

For each generated word from your **word** models, decide if it is a plausible word on English. You can poll friends or just use your best guess. Include your annotation in your tables, and for each model, say how many generated words were plausible.

Organise your tables to make it convenient to answer the following questions:

1. Does the plausibility of generated words get (monotone) higher or lower as n increases?

2. Does there seem to be an effect of corpus (train vs gutenber) on word plausibility?
3. Is there an effect of the tokeniser on generated word plausibiity?
4. What's your favourite generated word from any of your models?

For your **sentence** models, put your generated sentences into tables (or another convenient format) but don't explicitly evaluate them for grammaticality.

Answer the following questions:

1. For the sentences, why is it so hard to tell if the generated sentences are grammatical from the models trained on the **train.zip** dataset? Is it easier to tell with the Gutenberg corpus?
2. Do you think the sentences get (monotonically) more or less grammatical as n increases? If you see a pattern, why do you think it is? Of not, why do you think you don't see one?
3. There are some unexpected infinite perplexities with the Gutenberg corpus using its own **sents** method, but it is fixed when we use our own tokenizer with the **raw** method. What's going on here?

Checklist

Hand in a zip file containing:

1. **corpus_reader.py** containing:
 - **Tokenizer**
 - **TokenizerNLTK**
 - **CorpusReader**
2. **model.py** containing **NGramModel**
3. **generate.py** containing:
 - **generate_one**
 - **demonstrate**
 - script with command-line interface
4. **demo.py**
5. **tests.py** containing:
 - **test_tokenizer_plain**
 - **test_tokenize_nltk**
 - **test_ngram_table**
 - **test_probability_zero**
6. Report (no Word documents please!)